

# SQL COMMAND CENTER — VISTA DE BLOQUEOS (LOCK TREE)

Especificación para la Detección Jerárquica de Bloqueos y Modelo de Interfaz JavaFX

Este documento contiene la arquitectura de consulta recursiva y el modelo de clases necesario para integrar la detección e inspección jerárquica de bloqueos en **SQL Command Center**.

## SECCIÓN 1 Consulta SQL Recursiva para Detección de Bloqueos

La siguiente CTE recursiva cruza `pg_locks` y `pg_stat_activity` para identificar los procesos raíz bloqueadores y encadenar las sesiones bloqueadas dependientes:

```
WITH RECURSIVE lock_tree AS (
  -- Nodos raíz: Procesos bloqueadores iniciales
  SELECT
    blocking.pid AS blocking_pid,
    blocked.pid AS blocked_pid,
    1 AS depth,
    ARRAY[blocking.pid, blocked.pid] AS path
  FROM pg_locks blocked_locks
  JOIN pg_stat_activity blocked ON blocked.pid = blocked_locks.pid
  JOIN pg_locks blocking_locks
    ON blocking_locks.locktype = blocked_locks.locktype
    AND blocking_locks.database IS NOT DISTINCT FROM blocked_locks.database
    AND blocking_locks.relation IS NOT DISTINCT FROM blocked_locks.relation
    AND blocking_locks.page IS NOT DISTINCT FROM blocked_locks.page
    AND blocking_locks.tuple IS NOT DISTINCT FROM blocked_locks.tuple
    AND blocking_locks.transactionid IS NOT DISTINCT FROM blocked_locks.transactionid
    AND blocking_locks.pid != blocked_locks.pid
  JOIN pg_stat_activity blocking ON blocking.pid = blocking_locks.pid
  WHERE NOT blocked_locks.granted AND blocking_locks.granted

  UNION ALL

  -- Recursión: Encadenar bloqueos en cascada
  SELECT
    lt.blocked_pid AS blocking_pid,
    blocked.pid AS blocked_pid,
    lt.depth + 1 AS depth,
    lt.path || blocked.pid
  FROM lock_tree lt
  JOIN pg_locks blocked_locks ON blocked_locks.pid != lt.blocked_pid
  JOIN pg_stat_activity blocked ON blocked.pid = blocked_locks.pid
  JOIN pg_locks blocking_locks
    ON blocking_locks.pid = lt.blocked_pid
    AND blocking_locks.locktype = blocked_locks.locktype
    AND blocking_locks.granted
  WHERE NOT blocked_locks.granted AND NOT (blocked.pid = ANY(lt.path))
)
SELECT
  REPEAT('  L ', lt.depth - 1) || lt.blocked_pid::text AS visual_tree,
  lt.blocking_pid, lt.blocked_pid,
  blocker_act.username AS usuario_bloqueador,
  blocked_act.username AS usuario_bloqueado,
  blocked_act.wait_event_type, blocked_act.wait_event,
  ROUND(EXTRACT(EPOCH FROM (clock_timestamp() - blocked_act.state_change))::numeric, 2) AS tiempo_espera_seg,
  blocker_act.query AS consulta_bloqueadora,
  blocked_act.query AS consulta_bloqueada
FROM lock_tree lt
JOIN pg_stat_activity blocker_act ON blocker_act.pid = lt.blocking_pid
JOIN pg_stat_activity blocked_act ON blocked_act.pid = lt.blocked_pid
ORDER BY lt.path;
```

## SECCIÓN 2 Estructura del Modelo de Datos para JavaFX

### Mapeo del Nodo en Java (LockNode.java)

- **Campos clave:** `pid`, `usuario`, `eventoEspera`, `tiempoEsperaSeg`, `consulta` y `esRaiz`.
- **Componente recomendado:** Componente JavaFX `TreeTableView<LockNode>` para estructurar visualmente el árbol con sangría y colores diferenciados para los PIDs origen.